

Travel Fare Calculator

Case Study 173

Student	Stuti Saxena
Cohort	Larry Page
Roll Number	150096752024

Java Console Application

Prepared from the submitted project source files

1. Problem Statement

The Travel Fare Calculator is a Java console application designed to calculate the estimated travel cost for one or more travelers. The program collects traveler details, allows the user to select a destination and travel class, accepts the trip duration, applies the appropriate fare calculation and group discount, and finally displays a quotation.

The implementation is organized into separate Java classes so that traveler information, destination selection, travel class selection, and fare calculation have clear responsibilities.

2. Project Objective

The main objectives of the project are:

- Register multiple travelers by storing their name and age.
- Allow the user to choose a destination from the available options.
- Allow the user to choose Economy, Business, or First Class.
- Calculate the fare using destination fare, class multiplier, trip duration, and traveler count.
- Apply a group discount based on the number of travelers.
- Generate a clear quotation showing traveler and fare details.
- Allow the user to calculate another quotation after completing one quotation.

3. Project Flow

The program begins in Main.java. A Scanner object is created to read keyboard input. The program then asks for the number of travelers and creates a Traveler array of the required size.

- Traveler registration: a for loop collects the name and age of each traveler and creates a Traveler object.
- Destination selection: Destination.selectDestination() displays four choices and returns the selected destination.
- Travel class selection: TravelClass.selectClass() displays three choices and returns the selected class.
- Trip duration: the number of days is entered and validated.
- Fare calculation: the destination base fare and class multiplier are passed to FareCalculator.
- Discount: the number of travelers determines whether a 0%, 5%, or 10% group discount is applied.
- Quotation: Main.java prints the complete quotation, including traveler details and final cost.
- Repeat option: the while loop allows another quotation to be calculated.

4. Project Files and Responsibilities

The project is divided into five Java source files. Each file has a focused role.

File	Responsibility
Main.java	Controls program flow, input, traveler registration, calculations, quotation, and repetition.

File	Responsibility
Traveler.java	Represents a traveler using private name and age fields, a constructor, and getter methods.
Destination.java	Handles destination selection and provides the base fare for each destination.
TravelClass.java	Handles travel-class selection and provides the class multiplier.
FareCalculator.java	Contains methods for basic fare, discount, final fare, and discount-rate calculations.

5. Java Concepts Used

5.1 Classes and Objects

The project uses classes to organize related data and methods. The Traveler class is used to create traveler objects. In Main.java, a Traveler array stores the created objects.

5.2 Encapsulation

The Traveler class declares name and age as private fields. The values are assigned through the constructor and accessed using getName() and getAge() methods.

5.3 Constructor

Traveler(String name, int age) initializes a Traveler object with the supplied name and age.

5.4 Arrays

Main.java creates a Traveler[] array based on the number of travelers entered by the user. The for loop fills the array one traveler at a time.

5.5 Methods

The project uses separate methods for destination selection, class selection, fare calculation, discount calculation, final fare calculation, and discount-rate retrieval.

5.6 Scanner

java.util.Scanner is used to read keyboard input from the user.

5.7 switch Statement

switch is used for destination choices, destination base fares, travel-class choices, and class multipliers.

5.8 if-else

if-else is used to determine the group discount according to the number of travelers and to validate numeric ranges.

5.9 Loops

The project uses a for loop for registering multiple travelers and while loops for input validation and repeating quotations.

6. Module Details

6.1 Traveler Registration

Main.java asks for the number of travelers and creates a Traveler array. A for loop then asks for each traveler's name and age. Names are read with `nextLine()`, which allows names containing spaces. Age values less than or equal to zero are rejected and entered again.

6.2 Destination Selection

Destination.java displays Mumbai, Delhi, Goa, and Bangalore. The selected option is checked using a while loop and converted to the destination name using switch.

6.3 Travel Class

TravelClass.java provides three options: Economy, Business, and First Class. The class is selected using switch and its multiplier is returned by `getClassMultiplier()`.

6.4 Fare Calculation

FareCalculator.java performs the mathematical calculations. The basic fare uses destination fare, class multiplier, trip duration, and number of travelers.

6.5 Group Discount

The discount is based on group size. Five or more travelers receive a 10% discount, three or four travelers receive a 5% discount, and one or two travelers receive no discount.

6.6 Quotation Generation

Main.java displays the destination, travel class, duration, number of travelers, traveler details, base fare, class multiplier, total fare before discount, discount percentage, discount amount, and final travel cost.

7. Fare Structure Used

Destination	Base Fare / Traveler / Day
Mumbai	₹5,000
Delhi	₹7,000
Goa	₹3,000
Bangalore	₹4,500

Travel Class	Multiplier
Economy	1.0
Business	1.5
First Class	2.0

These fare values and multipliers are the values implemented in Destination.java and TravelClass.java.

8. Fare Calculation Formula

The program calculates the basic fare using the following formula:

$$\text{Basic Fare} = \text{Base Fare} \times \text{Class Multiplier} \times \text{Duration} \times \text{Number of Travelers}$$

The group discount is then calculated from the total fare before discount:

$$\text{Discount Amount} = \text{Total Fare} \times \text{Discount Rate}$$

Finally, the discount is subtracted from the total:

$$\text{Final Travel Cost} = \text{Total Fare} - \text{Discount Amount}$$

9. Requirement-to-Implementation Mapping

The following mapping connects the major project requirements with the actual implementation in the submitted source files.

Requirement	Implementation
Traveler registration	Main.java uses a Traveler[] array and a for loop to collect name and age for each traveler.
Destination selection	Destination.java uses menu choices, validation, and switch to select a destination.
Travel class	TravelClass.java uses menu choices, validation, and switch to select the travel class.
Fare calculation	FareCalculator.java uses methods and arithmetic operators to calculate the basic and final fare.
Group discount	FareCalculator.java uses if-else conditions for 0%, 5%, and 10% discounts.
Quotation generation	Main.java prints traveler details, fare inputs, discount information, and final travel cost.
Multiple quotations	Main.java uses a while loop and a Yes/No choice to repeat the quotation process.
Input validation	while loops validate destination, class, age, duration, and the repeat choice within their number of iterations.

10. Sample Execution Flow

A typical execution follows this sequence:

- The application displays the Travel Fare Calculator heading.
- The user enters the number of travelers.
- For each traveler, the user enters a name and age.
- The user selects a destination from four options.
- The user selects a travel class from three options.
- The user enters the trip duration in days.
- The application obtains the base fare and class multiplier.
- The application calculates the fare, discount, and final cost.
- A formatted trip quotation is displayed.
- The user chooses whether to calculate another quotation.

11. Input Validation and Error Handling

The submitted program contains simple range-based validation. A number of travelers less than or equal to zero is rejected. Age and trip duration are checked with while loops and must be greater than zero. Destination and travel-class choices must fall within their menu ranges. The repeat choice must be either 1 or 2.

The source code uses `Scanner.nextInt()` for numeric input. Therefore, the submitted version is designed for numeric entries at numeric prompts; it does not include try-catch based handling for non-numeric text.

12. Conclusion

The Travel Fare Calculator demonstrates a complete Java console application built from basic programming and object-oriented concepts. The program separates responsibilities across five classes, accepts multiple travelers, provides destination and class selection, calculates fares using methods and operators, applies group discounts using conditional statements, and presents the result as a quotation.

The implementation remains intentionally straightforward and uses the same project structure and logic as the submitted Java source files.

13. Submitted Source Files

- Main.java
- Traveler.java
- Destination.java
- TravelClass.java
- FareCalculator.java